Experiment 3:- Demonstrate the concepts of inheritance

Sr. No.	Topic	Leetcode id	Problem
1.	Inheritance	49	Group Anagrams
2.	Inheritance	125	Valid Palindrome *(inherit common string operations)*
3.	Inheritance	344	Reverse String *(inherit utility methods)*
4.	Inheritance	5	Longest Palindromic Substring

Homework -Demonstrate the concepts of inheritance

Sr. No.	Topic	Leetcode id	Problem
1.	Inheritance	14	Longest common prefix
2.	Inheritance	28	Find the Index of the First Occurrence in a String
3.	Inheritance	58	Length of Last Word
4.	Inheritance	67	Add Binary

3.1 Leetcode id- 49--- Group Anagrams

```
package Lab3;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class GroupAnagrams {

    public static void
main(String[] args) {
```

```
        String[] strs = {"eat",
"tea", "tan", "ate", "nat", "bat"};

        Solution obj = new
Solution();

        List<List<String>> result =
obj.groupAnagrams(strs);

        System.out.println("Grouped
Anagrams:");

        for (List<String> group :
result) {
System.out.println(group);
        }
    }
}

class Solution {

    public List<List<String>>
groupAnagrams(String[] strs) {
```

```
        List<List<String>> result =
new ArrayList<>();
        boolean[] visited = new
boolean[strs.length];

        for (int i = 0; i <
strs.length; i++) {

            if (visited[i]) {
                continue;
            }

            List<String> group =
new ArrayList<>();
            group.add(strs[i]);
            visited[i] = true;

            for (int j = i + 1; j <
strs.length; j++) {

                if (!visited[j] &&
isAnagram(strs[i], strs[j])) {
```

```
group.add(strs[j]);
                    visited[j] =
true;
                }
            }

            result.add(group);
        }

        return result;
    }

    public boolean isAnagram(String
s1, String s2) {

        if (s1.length() !=
s2.length()) {
            return false;
        }

        char[] a =
s1.toCharArray();
```

```
        char[] b =
s2.toCharArray();

        Arrays.sort(a);
        Arrays.sort(b);

        return Arrays.equals(a, b);
    }
}
```

3.2 Leetcode id- 125---Valid Palindrome

```
package Lab3;
import java.util.Scanner;

public class ValidPalindrome {

    public static void main(String[] args)
{
```

```
        Scanner sc = new
Scanner(System.in);

        System.out.print("Enter a string:
");
        String s = sc.nextLine();

        boolean result = isPalindrome(s);

        if (result) {
            System.out.println("The given
string is a Palindrome.");
        } else {
            System.out.println("The given
string is Not a Palindrome.");
        }

        sc.close();
    }

    public static boolean
isPalindrome(String s) {

        String str = "";

        // Remove special characters and
convert to lowercase
```

```
        for (int i = 0; i < s.length();
i++) {

            char ch = s.charAt(i);

            if
(Character.isLetterOrDigit(ch)) {
                str +=
Character.toLowerCase(ch);
            }
        }

        // Reverse the string
        String rev = "";

        for (int i = str.length() - 1; i
>= 0; i--) {
            rev += str.charAt(i);
        }

        // Compare original and reversed
strings
        return str.equals(rev);
    }
}
```

3.3 Leetcode id- 344--- Reverse String

```
import java.util.Scanner;

public class ReverseString {

    public static void main(String[] args)
{

        Scanner sc = new
Scanner(System.in);

        System.out.print("Enter a string:
");
        String input = sc.nextLine();

        // Convert the string to a
character array
        char[] s = input.toCharArray();

        // Call the method to reverse the
character array
        reverseString(s);
```

```
        System.out.print("Reversed String:
");
        System.out.println(new String(s));

        sc.close();
    }

    public static void
reverseString(char[] s) {

        char[] temp = new char[s.length];

        int index = 0;

        // Store characters in reverse
order
        for (int i = s.length - 1; i >= 0;
i--) {
            temp[index++] = s[i];
        }

        // Copy the reversed characters
back to the original array
        for (int i = 0; i < s.length; i++)
{
            s[i] = temp[i];
        }
```

```
    }
}
```

3.4 Leetcode id- 5--- Longest Palindromic Substring

```
package Lab3;


import java.util.Scanner;

public class LongestPalindrome {

    public static void
main(String[] args) {

        Scanner sc = new
Scanner(System.in);

        System.out.print("Enter a
string: ");
        String s = sc.nextLine();
```

```
        Solution1 obj = new
Solution1();

        String result =
obj.longestPalindrome(s);

        System.out.println("Longest
Palindromic Substring: " + result);

        sc.close();
    }
}

class Solution1{

    public String
longestPalindrome(String s) {

        String ans = "";

        // Check every possible
substring
```

```
        for (int i = 0; i <
s.length(); i++) {

            for (int j = i; j <
s.length(); j++) {

                // Extract
substring
                String sub =
s.substring(i, j + 1);

                // Check if it is a
palindrome
                if
(isPalindrome(sub) && sub.length()
> ans.length()) {
                    ans = sub;
                }
            }
        }

        return ans;
    }
```

```
    public boolean
isPalindrome(String str) {

        int left = 0;
        int right = str.length() -
1;

        while (left < right) {

            if (str.charAt(left) !=
str.charAt(right)) {
                return false;
            }

            left++;
            right--;
        }

        return true;
    }
}
```